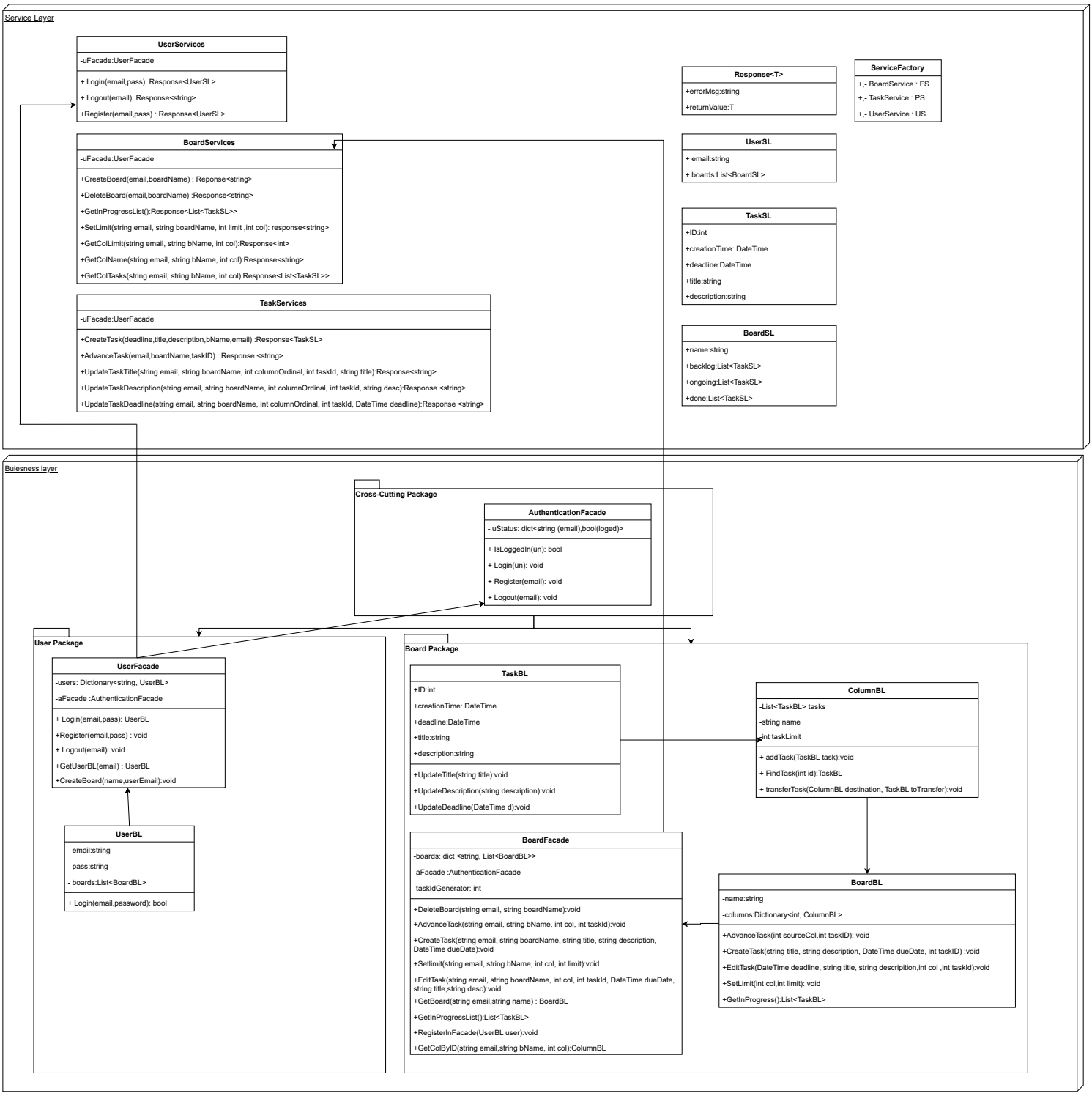

Service Layer
UserServices
-uFacade:UserFacade
+ Login(email,pass): Response<UserSL>
+ Logout(email): Response<string>
+Register(email,pass) : Response<UserSL>
BoardServices
-uFacade:UserFacade
+CreateBoard(email,boardName) : Reponse<string>
+DeleteBoard(email,boardName) :Response<string>
+GetInProgressList():Response<List<TaskSL>>
+SetLimit(string email, string boardName, int limit ,int col): response<string>
+GetColLimit(string email, string bName, int col):Response<int>
+GetColName(string email, string bName, int col):Response<string>
+GetColTasks(string email, string bName, int col):Response<List<TaskSL>>
TaskServices
-uFacade:UserFacade
+CreateTask(deadline,title,description,bName,email) :Response<TaskSL>
+AdvanceTask(email,boardName,taskID) : Response <string>
+UpdateTaskTitle(string email, string boardName, int columnOrdinal, int taskId, string title):Response<string>
+UpdateTaskDescription(string email, string boardName, int columnOrdinal, int taskId, string desc):Response <string>
+UpdateTaskDeadline(string email, string boardName, int columnOrdinal, int taskId, DateTime deadline):Response <string>
Response<T>
+errorMsg:string
+returnValue:T
ServiceFactory
+,- BoardService : FS
+,- TaskService : PS
+,- UserService : US
UserSL
+ email:string
+ boards:List<BoardSL>
TaskSL
+ID:int
+creationTime: DateTime
+deadline:DateTime
+title:string
+description:string
BoardSL
+name:string
+backlog:List<TaskSL>
+ongoing:List<TaskSL>
+done:List<TaskSL>
Buiesness layer
Cross-Cutting Package
AuthenticationFacade
- uStatus: dict<string (email),bool(loged)>
+ IsLoggedIn(un): bool
+ Login(un): void
+ Register(email): void
+ Logout(email): void
User Package
UserFacade
-users: Dictionary<string, UserBL>
-aFacade :AuthenticationFacade
+ Login(email,pass): UserBL
+Register(email,pass) : void
+ Logout(email): void
+GetUserBL(email) : UserBL
+CreateBoard(name,userEmail):void
UserBL
- email:string
- pass:string
- boards:List<BoardBL>
+ Login(email,password): bool
Board Package
TaskBL
+ID:int
+creationTime: DateTime
+deadline:DateTime
+title:string
+description:string
+UpdateTitle(string title):void
+UpdateDescription(string description):void
+UpdateDeadline(DateTime d):void
ColumnBL
-List<TaskBL> tasks
-string name
-int taskLimit
+ addTask(TaskBL task):void
+ FindTask(int id):TaskBL
+ transferTask(ColumnBL destination, TaskBL toTransfer):void
BoardFacade
-boards: dict <string, List<BoardBL>>
-aFacade :AuthenticationFacade
-taskIdGenerator: int
+DeleteBoard(string email, string boardName):void
+AdvanceTask(string email, string bName, int col, int taskId):void
+CreateTask(string email, string boardName, string title, string description, DateTime dueDate):void
+Setlimit(string email, string bName, int col, int limit):void
+EditTask(string email, string boardName, int col, int taskId, DateTime dueDate, string title,string desc):void
+GetBoard(string email,string name) : BoardBL
+GetInProgressList():List<TaskBL>
+RegisterInFacade(UserBL user):void
+GetColByID(string email,string bName, int col):ColumnBL
BoardBL
-name:string
-columns:Dictionary<int, ColumnBL>
+AdvanceTask(int sourceCol,int taskID): void
+CreateTask(string title, string description, DateTime dueDate, int taskID) :void
+EditTask(DateTime deadline, string title, string descripition,int col ,int taskId):void
+SetLimit(int col,int limit): void
+GetInProgress():List<TaskBL>